

Akademia Wojsk Lądowych
im. gen. Tadeusza Kościuszki we Wrocławiu

Sprawozdanie z projektu semestralnego
Przedmiot: **Logika Układów Cyfrowych**

Macierz sensoryczna

z zastosowaniem sensorów pogodowych

Platforma: **Zybo Z7-20** | Język: **Verilog** | Narzędzie: **Vivado 2025.2**

Przygotowali:

Wiktor Zieliński	nr albumu: 11854
Wiktor Dams	nr albumu: 11864
Kamil Mania	nr albumu: 11137

Projekt prowadzony przez:
Mgr. Inż. Igor Mielczarek

Wrocław, 2026

Spis treści

1	Założenia systemu	1
1.1	Wymagania techniczne	1
1.2	Wymagania funkcjonalne	2
1.3	Ograniczenia i przyjęte założenia projektowe	2
2	Zastosowanie systemu	4
2.1	Potencjalne obszary użycia	4
2.2	Spełnienie wymagań i efektywność działania	4
2.3	Konkretny przypadek użycia	5
2.4	Demonstracja działania systemu	5
3	Implementacja systemu	8
3.1	Architektura ogólna	8
3.2	Multipleksery	9
3.2.1	Multiplekser wyboru trybu wyświetlania	10
3.2.2	Multiplekser wyboru bajtu w kontrolerze I2C	10
3.2.3	Multiplekser kolorów pikseli w sterowniku OLED	11
3.3	Dekodery	12
3.4	Enkodery	13
3.5	Maszyny stanów (FSM)	15
3.5.1	Typ automatów - Moore vs Mealy	15
3.5.2	FSM kontrolera I2C	15
3.5.3	FSM sterownika BME680	17
3.5.4	FSM sterownika CCS811	17
3.5.5	FSM sterownika wyświetlacza OLED	18
3.6	Rejestry	20
3.6.1	Rejestr przesuwany PISO w nadajniku SPI	20
3.6.2	Rejestr przesuwany SIPO w odbiorze I2C	21
3.6.3	Rejestr synchronizacyjny (metastabilność)	21
3.6.4	Bufor tekstowy	22
3.7	Implementacja protokołu I2C	23
3.7.1	Zasada działania open-drain	23
3.7.2	Generowanie warunków START i STOP	23
3.7.3	Monolityczna transakcja - klucz do poprawności	24
3.7.4	Przeliczanie wartości surowych BME680	25
3.8	Diagnostyka Pmod AQS (CCS811)	26

3.8.1	Objaw początkowy	26
3.8.2	Pierwsza hipoteza i jej weryfikacja	26
3.8.3	Wykrycie i naprawa błędu w kontrolerze I2C	26
3.8.4	Diagnostyka sprzętowa CCS811	27
3.8.5	Wniosek diagnostyczny	27
4	Wybrane fragmenty kodu źródłowego	29
4.1	Moduł nadrzędny - obsługa linii I2C i wybór sygnałów	29
4.2	Kontroler I2C - wysyłanie adresu i bajtu danych	30
4.3	Sterownik BME680 - konfiguracja i odczyt surowych danych	31
4.4	Formater tekstu - multiplekser trybów i konwersja cyfr	32
4.5	Sterownik OLED - inicjalizacja i renderowanie pikseli	33
4.6	Plik ograniczeń XDC - przypisanie pinów i ochrona OLED	34
5	Podsumowanie	36
6	Bibliografia	37

1 Założenia systemu

Celem projektu jest budowa macierzy sensorycznej realizującej funkcję stacji pogodowej na płycie FPGA Zybo Z7-20. System zbiera dane środowiskowe przy użyciu dwóch niezależnych czujników podłączonych do magistrali I2C i prezentuje wyniki w czasie rzeczywistym na wyświetlaczu Pmod OLED RGB. Użytkownik steruje trybem wyświetlania za pomocą przełączników suwakowych.

1.1 Wymagania techniczne

Projekt nakłada następujące wymagania sprzętowe i protokołowe:

- Platforma sprzętowa: Zybo Z7-20 z układem Zynq-7000, zegar systemowy 125 MHz.
- Sensor BME680 (SparkFun Qwiic) - pomiar temperatury, wilgotności względnej i ciśnienia atmosferycznego; komunikacja I2C, adres 0x77, port Pmod JC.
- Pmod AQS (Digilent) z układem CCS811 - pomiar ekwiwalentnego stężenia CO₂ (eCO2) i lotnych związków organicznych (TVOC); komunikacja I2C, adres 0x5B, port Pmod JD.
- Wyświetlacz Pmod OLED RGB oparty na sterowniku SSD1331 - matryca 96×64 pikseli, kolor 16-bitowy RGB565; komunikacja SPI, port Pmod JE.
- Cztery przełączniki suwakowe (SW0–SW3) oraz cztery diody LED do sterowania i sygnalizacji stanu systemu.
- Standard napięciowy 3,3 V LVCMOS33 dla wszystkich sygnałów peryferyjnych.

Na poziomie komunikacji przyjęto następujące założenia protokołowe. Magistrala I2C pracuje w trybie standardowym (100 kHz). Ze względu na brak wspólnej magistrali (oba sensory różnią się zarówno adresem, jak i sekwencją inicjalizacji) zdecydowano o rozdzieleniu komunikacji na dwie niezależne magistrale - każda obsługiwana przez osobny egzemplarz kontrolera I2C. Magistrala SPI dla wyświetlacza pracuje z częstotliwością zegara 6,25 MHz.

1.2 Wymagania funkcjonalne

System musi realizować następujące funkcjonalności dostępne dla użytkownika:

- Przełącznik SW3 pełni funkcję głównego włącznika logiki systemu. Stan niski powoduje wyłączenie wyświetlacza i zatrzymanie komunikacji z sensorami.
- Tylko SW0 w górę - wyświetlanie aktualnej wilgotności względnej z sensora BME680 (np. 45,9 %).
- Tylko SW1 w górę - wyświetlanie aktualnego ciśnienia atmosferycznego z sensora BME680 (np. 1011 hPa).
- Oba przełączniki SW0 i SW1 w górę - wyświetlanie danych z Pmod AQS: stężenia eCO2 [ppm] i TVOC [ppb].
- Oba przełączniki SW0 i SW1 w dół - wyświetlanie aktualnej temperatury z sensora BME680 (np. 22,1 °C).
- Dane na wyświetlaczu odświeżane są co 500 ms - dając efekt podglądu warunków w czasie rzeczywistym.
- Diody LED sygnalizują stan systemu: LD0 - aktywność (SW3), LD1 - inicjalizacja OLED'a zakończona, LD2 - BME680 odpowiada, LD3 - Pmod AQS odpowiada.

1.3 Ograniczenia i przyjęte założenia projektowe

W projekcie przyjęto szereg założeń upraszczających wynikających z dostępności sprzętu i zakresu przedmiotu:

- Przeliczenie wartości surowych z sensora BME680 na jednostki fizyczne opiera się na kalibracji empirycznej - zamiast pełnego algorytmu kompensacyjnego Boscha (wymagającego odczytu 23 rejestrów kalibracyjnych z pamięci OTP), zastosowano wzory liniowe dopasowane do konkretnego egzemplarza sensora. Wzory te są dokładne z odpowiedzialnością ± 2 °C, $\pm 5\%$ RH, ± 5 hPa.
- Wszystkie obliczenia wykonywane są w arytmetyce całkowitej (stało-przecinkowej) bez użycia jednostki zmiennoprzecinkowej.

- System nie przechowuje historii pomiarów, każde odświeżenie ekranu prezentuje bieżącą wartość.
- Czcionka znaków zdefiniowana jest statycznie jako pamięć ROM 8×8 pikseli na znak i obejmuje wyłącznie znaki potrzebne w projekcie.
- W trakcie realizacji projektu stwierdzono uszkodzenie sprzętowe egzemplarza Pmod AQS, układ CCS811 zwracał rejestr HW_ID o wartości 0x00 zamiast fabrycznej stałej 0x81. Sterownik CCS811 jest zaimplementowany zgodnie z dokumentacją i gotowy do pracy ze sprawnym egzemplarzem; w bieżącej wersji tryb AQS wyświetla diagnostykę w postaci surowych wartości szesnastkowych i kodu statusu.

2 Zastosowanie systemu

Zrealizowany system stanowi kompletną, autonomiczną stację monitoringu warunków środowiskowych zbudowaną na rekonfigurowalnym układzie FPGA. W odróżnieniu od rozwiązań mikrokontrolerowych, architektura sprzętowa zapewnia deterministyczny, równoległy odczyt z wielu sensorów bez konieczności stosowania systemu operacyjnego czasu rzeczywistego.

2.1 Potencjalne obszary użycia

Proponowane zastosowania obejmują kilka dziedzin praktycznych:

- Stacja meteorologiczna wewnętrzna i zewnętrzna - ciągły pomiar temperatury, wilgotności i ciśnienia z wizualizacją na wbudowanym wyświetlaczu. System może służyć jako samodzielne urządzenie wyświetlające warunki atmosferyczne w pomieszczeniu, serwerowni lub na zewnątrz budynku (po zabezpieczeniu obudową).
- Monitoring jakości powietrza w pomieszczeniach - dzięki sensorowi Pmod AQS (CCS811) system może wykrywać podwyższone stężenia CO₂ i związków organicznych (TVOC), co jest istotne w biurach, szkołach, laboratoriach chemicznych i domach.
- Inteligentny budynek i automatyka domowa - odczyty ze stacji mogą być podstawą automatycznego sterowania wentylacją, klimatyzacją i ogrzewaniem. Platforma FPGA umożliwia łatwą rozbudowę o dodatkowe interfejsy wyjściowe.
- Prototyp platformy badawczej - architektura modułowa (każdy sensor obsługiwany przez niezależny kontroler I2C) umożliwia łatwe dodawanie kolejnych czujników bez modyfikacji istniejącego kodu.

2.2 Spełnienie wymagań i efektywność działania

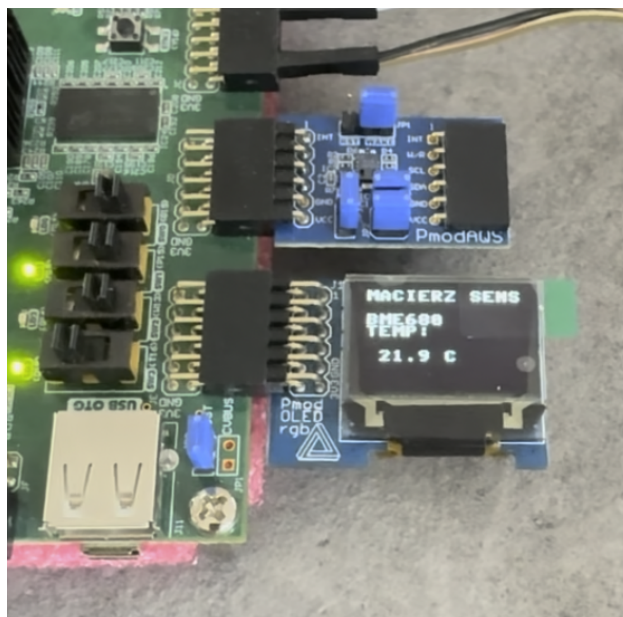
System spełnia wymagania projektu w zakresie akwizycji danych w czasie rzeczywistym. Cykl pomiarowy BME680 wynosi w przybliżeniu 300 ms (50 ms na wykonanie pomiaru przez sensor oraz 250 ms oczekiwania między cyklami), co zapewnia aktualizację danych kilka razy na sekundę. Wyświetlacz odświeżany jest co 500 ms, co jest wystarczające dla wartości zmieniających się powoli, takich jak temperatura czy ciśnienie. Dwie niezależne magistrale

I2C, każda taktowana zegarem 100 kHz - działają równolegle i nie blokują się wzajemnie, co jest istotną zaletą architektury FPGA względem rozwiązań mikrokontrolerowych, w których obie magistrale musiałyby być obsługiwane sekwencyjnie przez jeden procesor. Obsługa wyświetlacza SPI jest całkowicie asynchroniczną względem akwizycji danych sensorowych.

2.3 Konkretny przypadek użycia

Typowy scenariusz pracy systemu: operator włącza zasilanie płytki Zybo Z7-20 i ustawia przełącznik SW3 w górę. Przez pierwsze 300–400 ms inicjalizowany jest wyświetlacz OLED, czego dowodem jest kolejne zapalenie się diod LD0, następnie LD1. System nawiązuje komunikację z sensorami, diody LD2 i LD3 informują o poprawności połączenia I2C. Na ekranie pojawia się aktualny pomiar. Operator może w dowolnym momencie zmienić wyświetlaną wielkość przez przestawienie przełączników SW0/SW1, a ekran zostaje odświeżony natychmiast po zmianie trybu. Użycie przycisku BTN0 powoduje reset i ponowną inicjalizację systemu.

2.4 Demonstracja działania systemu



Rysunek 1: Odczyt temperatury z sensora BME680



Rysunek 2: Odczyt ciśnienia z sensora BME680



Rysunek 3: Odczyt wilgotności z sensora BME680

3 Implementacja systemu

System zaimplementowano w języku opisu sprzętu Verilog w narzędziu Vivado w wersji przeznaczonej do programowania układu Zynq-7020 na płycie Zybo Z7-20. Całość składa się z siedmiu modułów hierarchicznie połączonych przez moduł nadrzędny. Poniżej omówiono każdą z kluczowych struktur logiki cyfrowej zastosowanych w projekcie.

3.1 Architektura ogólna

Moduł nadrzędny `weather_matrix_top` tworzy dwie niezależne ścieżki komunikacji I2C oraz jedną ścieżkę SPI dla wyświetlacza. Dane przepływają w jednym kierunku: sterowniki sensorów dostarczają wartości liczbowe do formatera tekstu, który wypełnia bufor tekstowy, a sterownik wyświetlacza renderuje zawartość bufora na ekranie.

Tabela 1: Moduły Verilog projektu oraz ich role

Moduł	Rola	Protokół / interfejs
i2c_master ($\times 2$)	Kontroler magistrali szeregowej	I2C 100 kHz
bme680_driver	Sterownik sensora pogodowego	I2C slave 0x77
ccs811_driver	Sterownik sensora jakości powietrza	I2C slave 0x5B
display_formatter	Budowanie bufora tekstowego	Wewnętrzny bus
oled_final	Renderowanie tekstu i sterowanie OLED	SPI 6,25 MHz
font_rom	Pamięć ROM znaków 8×8	Kombinacyjna
weather_matrix_top	Moduł nadrzędny łączący wszystkie powyższe moduły	Wszystkie

3.2 Multiplexery

Multiplexer (w skrócie MUX) to układ kombinacyjny, który działa jak przełącznik sterowany cyfrowo - spośród wielu sygnałów wejściowych wybiera dokładnie jeden i przepuszcza go na wyjście. Wyboru dokonuje się za pomocą dodatkowych wejść sterujących zwanych liniami selekcji. Multiplexer o n liniach selekcji obsługuje 2^n wejść danych - przykładowo 2-bitowy sygnał mode pozwala wybrać spośród 4 źródeł (multiplexer 4:1). W logice cyfrowej multiplexery są fundamentalnym budulcem wszędzie tam, gdzie wiele źródeł sygnału rywalizuje o jedno wyjście - od selekcji danych wyświetlanych na ekranie, przez wybór bajtu do wysłania przez magistralę, aż po routing sygnałów wewnątrz układu FPGA. W projekcie zastosowano multi-

pleksery w kilku miejscach, realizując selekcję sygnałów na poziomie danych i adresów.

3.2.1 Multiplekser wyboru trybu wyświetlania

Centralny multiplekser czterowejściowy zawarty w module `display_formatter.v` dokonuje wyboru wyświetlanej wielkości fizycznej na podstawie dwubitowego sygnału `mode`, który powstaje z połączenia przełączników SW1 i SW0:

Listing 1: Multiplekser wyboru trybu wyswietlania

```
1 wire enable      = sw_sync_2[3];      // SW3: włącznik
   glowny
2 wire [1:0] mode = sw_sync_2[1:0]; // SW1:SW0: wybor
   trybu
3 case (mode)
4     2'b00: // temperatura BME680
5     2'b01: // wilgotnosc BME680
6     2'b10: // cisnienie BME680
7     2'b11: // Pmod AQS (eCO2 + TVOC) lub diagnostyka RAW
           HEX
8 endcase
```

Jest to klasyczny multiplekser 4:1, gdzie sygnał wyboru (2-bitowy tryb) determinuje, które dane trafią do bufora tekstowego w danym cyklu aktualizacji. Realizacja poprzez instrukcję `case` w blok sekwencyjny (`always @(posedge clk)`) zapewnia synchroniczne przełączanie trybów bez ryzyka konfliktu sygnałów.

3.2.2 Multiplekser wyboru bajtu w kontrolerze I2C

Moduł `i2c_master` zawiera 8-wejściowy multiplekser kombinacyjny, który na podstawie 4-bitowego licznika bajtów `byte_idx` wybiera odpowiedni bajt z bufora zapisu do nadania przez rejestr przesuwany:

Listing 2: Multiplekser wyboru bajtu w kontrolerze I2C

```
1 reg [7:0] wr_sel;  
2 always @* begin  
3     case (byte_idx)  
4         4'd0: wr_sel = wr_data0;  
5         4'd1: wr_sel = wr_data1;  
6         // ...  
7         default: wr_sel = wr_data7;  
8     endcase  
9 end
```

Multiplekser ten jest blokiem czysto kombinacyjnym (always @*), wartość wr_sel zmienia się natychmiastowo po zmianie byte_idx. Dzięki temu maszyna stanów może załadować nowy bajt do rejestru przesuwneego w jednym cyklu zegara po inkrementacji licznika.

3.2.3 Multiplekser kolorów pikseli w sterowniku OLED

W module oled_final.v dla każdego piksela podczas renderowania wykonywany jest wybór między kolorem tekstu a kolorem tła na podstawie odpowiedniego bitu wzorca czcionki:

Listing 3: Multiplekser kolorow pikseli w sterowniku OLED

```
1 // Wybór koloru gornego bajtu RGB565:  
2 if (pixel_byte[3'd7 - pix_col[2:0]])  
3     shift_reg <= TXT_HI; // piksel tekstu: 0xFF  
4 else  
5     shift_reg <= BG_HI; // tlo: 0x00
```

Jest to multiplekser 2:1, gdzie sygnałem wyboru jest jeden bit wzorca znaku, a wyjściem jeden z dwóch predefiniowanych kolorów. Operacja ta powtarzana jest dla każdego z 64 pikseli każdego znaku (8×8).

3.3 Dekodery

Dekoder to układ kombinacyjny realizujący mapowanie jeden-do-jednego między zakodowanym wejściem a aktywnym wyjściem. Przyjmuje n -bitowe słowo wejściowe i aktywuje dokładnie jedno spośród 2^n wyjść odpowiadające danej kombinacji bitów. W systemach cyfrowych dekodery pojawiają się wszędzie tam, gdzie zakodowany adres lub indeks musi zostać przetłumaczony na bezpośredni sygnał sterujący - np. wybór wiersza pamięci, identyfikacja znaku do wyświetlenia, czy sterowanie konkretnym peryferykiem. Ważną cechą dekodera jest to, że jest układem czysto kombinacyjnym - nie posiada pamięci, a jego wyjście zmienia się natychmiastowo wraz ze zmianą wejścia. W module `font_rom.v` zastosowano dekodery znaków ASCII. Blok kombinacyjny `case` mapuje 8-bitowy kod znaku (wejście) na 64-bitowy wzorek graficzny (wyjście). Jest to przykład dekodera z pełnym rozwinięciem przypadków:

Listing 4: Dekoder znaku ASCII w `font_rom`

```
1 always @* begin
2     case (char_code)
3         8'h41: glyph = 64'h18_3C_66_66_7E_66_66_00; // '
3             A '
4         8'h42: glyph = 64'h7C_66_66_7C_66_66_7C_00; // '
4             B '
5         8'h30: glyph = 64'h3C_66_6E_76_66_66_3C_00; //
5             '0'
6         // ...
7         default: glyph = 64'h00_00_00_00_00_00_00_00; //
7             nieznany znak
8     endcase
9 end
```

Drugi blok kombinacyjny w tym samym module pełni funkcję dekodera wyboru wiersza - mapuje 3-bitowy numer wiersza (0–7) na odpowiedni 8-bitowy fragment 64-bitowego wzorca:

Listing 5: Dekoder wyboru wiersza w *font_rom*

```

1 always @* begin
2     case (row)
3         3'd0: pixels = glyph[63:56];
4         3'd1: pixels = glyph[55:48];
5         // ...
6         3'd7: pixels = glyph[7:0];
7     endcase
8 end

```

3.4 Enkodery

Enkoder to układ kombinacyjny działający odwrotnie do dekodera - przekształca sygnał z formatu szerokiego (wiele linii) na reprezentację zwartą (mniej bitów). Klasyczny enkoder priorytetowy z 2^n wejść produkuje n -bitowy kod wskazujący aktywne wejście o najwyższym priorytecie. W szerszym sensie enkodery obejmują wszelkie przetworniki formatu: konwersję wartości binarnej na kod ASCII, zamianę liczby na cyfry dziesiętne, czy kodowanie Gray'a. Kluczową właściwością enkodera jest kompresja informacji - reprezentacja danych w postaci wygodniejszej do dalszego przetwarzania lub transmisji. W module *display_formatter.v* zaimplementowano enkoder wartości czterobitowej na kod znaku szesnastkowego ASCII. Funkcja *hex_char* realizuje konwersję nibble'a (4 bity, wartość 0–15) na odpowiadający mu znak '0'–'9' lub 'A'–'F':

Listing 6: Enkoder *hex_char* – konwersja nibble na ASCII

```

1 function [7:0] hex_char;
2     input [3:0] nibble;
3     begin
4         if (nibble < 4'd10)
5             hex_char = 8'h30 + {4'b0, nibble}; // znaki
6                                     '0'..'9'
7         else
8             hex_char = 8'h41 + {4'b0, nibble} - 8'd10;
9                                     // znaki 'A'..'F'
10    end
11 endfunction

```


Funkcja ta jest blokiem czysto kombinacyjnym, bez pamięci - realizuje klasyczną funkcję enkodera przekształcającego kod binarny na kod ASCII. Stosowana jest w trybie diagnostycznym (tryb 11 SW0 i SW1 do góry) do wyświetlania surowych wartości rejestrów sensorów w zapisie szesnastkowym, co umożliwia weryfikację poprawności komunikacji I2C.

3.5 Maszyny stanów (FSM)

Maszyna stanów skończonych (ang. *Finite State Machine*, FSM) to model obliczeniowy układu sekwencyjnego, w którym zachowanie systemu zależy nie tylko od bieżących wejść, ale też od jego historii - zakodowanej w postaci aktualnego stanu. FSM składa się ze skończonego zbioru stanów, funkcji przejść (które stany następują po sobie przy danych wejściach) i funkcji wyjść. W odróżnieniu od układów kombinacyjnych, FSM posiada pamięć - rejestr stanu przechowujący informację o tym, w którym miejscu sekwencji operacji system się aktualnie znajduje. To czyni FSM naturalnym narzędziem do implementacji protokołów komunikacyjnych, sekwencji inicjalizacji i wszelkich procesów, które mają określony porządek kroków. W Verilogu FSM implementuje się typowo jako blok `always @(posedge clk)` z instrukcją `case` na rejestrze stanu.

3.5.1 Typ automatów - Moore vs Mealy

Wyróżnia się dwa podstawowe typy FSM różniące się sposobem generowania wyjść. W automacie Moore'a wartości wyjść zależą wyłącznie od bieżącego stanu - niezależnie od wejść. W automacie Mealy'ego wyjścia zależą jednocześnie od stanu i od aktualnych wejść, co pozwala zredukować liczbę potrzebnych stanów, ale wprowadza kombinacyjną ścieżkę między wejściem a wyjściem, grożącą hazardami. Wszystkie cztery FSM zastosowane w niniejszym projekcie zaimplementowano jako automaty Moore'a - sygnały sterujące liniami I2C (`sda_oe`, `scl_oe`) oraz SPI (`send_request`) są wystawiane z rejestru stanu, nie z logiki kombinacyjnej.

3.5.2 FSM kontrolera I2C

Kontroler I2C implementuje monolityczną maszynę stanów 14-stanową, realizującą pełną transakcję I2C od warunku START do warunku STOP bez przerywania magistrali. Jest to kluczowe wymaganie protokołu I2C - zwolnienie linii w trakcie transakcji generuje fałszywy warunek STOP i przerywa komunikację.

Tabela 2: Stany maszyny i ich przeznaczenie

Stan	Opis działania
ST_IDLE	Oczekiwanie na sygnał start; zerowanie sygnałów magistrali
ST_START	Generowanie warunku START: SDA opada przy wysokim SCL
ST_ADDR_W	Wysyłanie 8 bitów adresu slave + bit kierunku W (8 taktów SCL)
ST_ADDR_ACK	Odczyt bitu ACK od slave po adresie; próbkowanie w ćwiartce $q=2$
ST_WR_BYTE	Wysyłanie bajtu danych bit po bicie (MSB naprzód)
ST_WR_ACK	Odczyt ACK po bajcie; warunek przejścia do kolejnego bajtu lub STOP/RESTART
ST_WR_LOAD	1 cykl oczekiwania na propagację nowego <code>byte_idx</code> przez multiplekser
ST_RESTART	Generowanie powtórnego warunku START (dla transakcji odczytu)
ST_ADDR_R	Wysyłanie adresu slave z bitem kierunku R
ST_ADDR_R_ACK	Odczyt ACK po adresie+R
ST_RD_BYTE	Odbiór bajtu danych; wsuwanie bitów do rejestru przesuwne
ST_RD_ACK	Wysyłanie ACK lub NACK
ST_STOP	Generowanie warunku STOP: SDA rośnie przy wysokim SCL
ST_DONE	Sygnalizacja zakończenia; reset sygnałów; powrót do IDLE

Taktowanie bitów realizowane jest przez wewnętrzny generator ćwiartek okresu SCL (sygnał q, wartości 0–3). Każdy bit trwa dokładnie cztery ćwiartki: q=0 to faza przygotowania SDA przy niskim SCL, q=1 i q=2 to faza wysokiego SCL (w trakcie q=1 slave próbkuję linię danych lub master próbkuję ACK), q=3 to faza opadającego zbocza SCL.

3.5.3 FSM sterownika BME680

Sterownik realizuje cykliczny protokół pomiaru sensora w 10 stanach:

Listing 7: Stany maszyny FSM sterownika BME680

```

1 localparam S_IDLE      = 4'd0, // oczekiwanie na enable
2      S_WR_HUM          = 4'd1, // zlecenie: WRITE reg 0
      x72 = 0x01
3      S_WR_HUM_W        = 4'd2, // oczekiwanie na done
4      S_WR_MEAS         = 4'd3, // zlecenie: WRITE reg 0
      x74 = 0x25 (forced mode)
5      S_WR_MEAS_W       = 4'd4,
6      S_MEAS_DELAY      = 4'd5, // odczekaj 50 ms (czas
      pomiaru sensora)
7      S_RD              = 4'd6, // zlecenie: READ 8
      bajtow od reg 0x1F
8      S_RD_W            = 4'd7,
9      S_PROCESS         = 4'd8, // przelicz raw na C /
      proc / hPa
10     S_CYCLE_WAIT      = 4'd9; // odczekaj 250 ms,
      powtorz

```

Kluczowym elementem jest stan S_PROCESS, w którym sterownik składa bajty surowych danych w 20-bitowe (ciśnienie, temperatura) i 16-bitowe (wilgotność) wartości całkowite, a następnie przelicza je na jednostki fizyczne za pomocą operacji arytmetycznych. Sterownik sprawdza też poprawność odczytu, jeśli raw_temp wynosi 0xFFFFF (wszystko to jedyne, co oznacza brak odpowiedzi sensora na I2C), wyniki ustawiają się na zero.

3.5.4 FSM sterownika CCS811

Sterownik CCS811 jest bardziej złożony od BME680 z uwagi na wymaganą sekwencję inicjalizacji. Składa się z 22 stanów podzielonych na dwie fazy: Faza inicjalizacji (jednorazowa):

- S_IDLE → S_WAKE: wybudzenie sensora sygnałem nWAKE i odczekanie 100 ms czasu rozruchu bootloadera.
- S_RD_HWID: odczyt rejestru identyfikacyjnego (HW_ID, adres 0x20). Sprawny CCS811 zwraca zawsze wartość 0x81, jest to fabryczna stała zabezpieczona przed zapisem. Wynik trafia do rejestru diagnostycznego dbg_raw0.
- S_RD_ST1 / S_CHK1: odczyt rejestru STATUS (adres 0x00) i sprawdzenie bitu 7 (FW_MODE). Bit równy 1 oznacza, że aplikacja pomiarowa już działa.
- S_APP: jeśli FW_MODE = 0, wysłanie komendy APP_START (zapis wskaźnika rejestru 0xF4 bez bajtu danych). Sterownik ponawia próbę do 5 razy z weryfikacją statusu między próbami.
- S_MEAS: konfiguracja trybu pomiaru - zapis rejestru MEAS_MODE (0x01) wartością 0x10 (tryb 1: pomiar co 1 sekundę).

Faza pomiaru (cykliczna):

- S_RD_STAT: cykliczny odczyt rejestru STATUS i sprawdzenie bitu 3 (DATA_READY).
- S_RD_DATA: jeśli dane są gotowe, odczyt rejestru ALG_RESULT_DATA (adres 0x02), 4 bajty: [eCO2_hi, eCO2_lo, TVOC_hi, TVOC_lo].
- S_PROCESS: złożenie bajtów w 16-bitowe wartości eCO2 i TVOC operacją konkatencji.

3.5.5 FSM sterownika wyświetlacza OLED

Sterownik OLED implementuje największą maszynę stanów w projekcie - 28 stanów - obejmującą sekwencyjną inicjalizację i cykliczne renderowanie tekstu. Faza inicjalizacji (jednorazowa):

- S_OFF: oczekiwanie na enable; wszystkie sygnały OLED w stanie nieaktywnym.
- S_PMODEN_DLY: włączenie sygnału PMODEN (zasilanie logiki sterownika SSD1331) i odczekanie 50 ms czasu stabilizacji.

- S_RES_LO / S_RES_HI: generowanie impulsu zerującego (reset) układu sterującego (5 ms w stanie niskim, 5 ms w stanie wysokim).
- S_INIT_START/BYTE/NEXT: wysyłanie przez SPI kolejnych bajtów z tablicy init_rom (38 bajtów konfiguracyjnych dla SSD1331).
- S_POST_INIT / S_VCCEN_DLY: włączenie sygnału VCCEN (wysokie napięcie dla matrycy OLED) i odczekanie 100 ms. Kolejność jest krytyczna, wysokie napięcie musi być włączone dopiero po skonfigurowaniu sterownika.
- S_DISPON_START/WAIT: wysłanie komendy Display ON (0xAF) przez SPI.
- S_CLR_START/WAIT/NEXT: wysłanie komendy Clear Window (5 bajtów) - wyczyszczenie ekranu.
- S_READY: wyświetlacz gotowy; sygnał ready = 1; oczekiwanie na impuls refresh.

Faza renderowania (wyzwalana cyklicznie sygnałem refresh):

- S_CHAR_ADDR/LATCH: wystawienie adresu bufora tekstowego i zatrzaśnięcie odczytanego kodu znaku ASCII w rejestrze font_char po 3 cyklach latencji.
- S_WIN_START/WAIT/NEXT: wysłanie 6 bajtów komend ustawiających okno rysowania (8×8 pikseli) dla bieżącego znaku na ekranie.
- S_PIX_PREP/HI/LO/NEXT: dla każdego z 64 pikseli znaku - odczyt bitu wzorca z font_rom, wybór koloru (tekst/tło) i wysłanie dwóch bajtów RGB565 przez SPI.
- S_NEXT_CHAR: przejście do kolejnego znaku (12 kolumn × 8 wierszy = 96 znaków łącznie).

3.6 Rejestry

Rejestr to podstawowy element pamięci w logice sekwencyjnej - zbiór przerzutników D zatrzymujących dane synchronicznie z zegarem. Pojedynczy przerzutnik D zapamiętuje 1 bit; rejestr n -bitowy to n przerzutników sterowanych wspólnym zegarem. W odróżnieniu od układów kombinacyjnych, rejestr przechowuje stan - jego wyjście nie zmienia się między zboczami zegara, co daje pełną kontrolę nad momentem, w którym dane są aktualizowane. W systemach FPGA rejestry pełnią funkcje od prostego zatrzymywania wartości, przez bufor danych między modułami, aż po rejestry przesuwne realizujące serializację i deserializację danych. Szczególnie istotne typy to:

- **PISO** (*Parallel In, Serial Out*) - dane ładowane równolegle (cały bajt naraz), a następnie wysuwane bit po bicie; stosowany w nadajnikach SPI
- **SIPO** (*Serial In, Parallel Out*) - bity wsuwane szeregowo (jeden po drugim z linii danych), gromadzone aż do skompletowania pełnego słowa; stosowany w odbiornikach I2C i SPI
- **PIPO** (*Parallel In, Parallel Out*) - dane wchodzi i wychodzą równolegle; stosowany jako rejestr synchronizacyjny eliminujący metastabilność sygnałów zewnętrznych

W projekcie zastosowano kilka rodzajów rejestrów, realizujących różne funkcje logiczne.

3.6.1 Rejestr przesuwany PISO w nadajniku SPI

Sterownik OLED zawiera wbudowany nadajnik SPI oparty na rejestrze przesuwany PISO (ang. Parallel In, Serial Out - wejście równoległe, wyjście szeregowe). Bajt danych ładowany jest równoległe do rejestru `shift_reg`, a następnie wysuwany bit po bicie na linię MOSI, najstarszym bitem naprzód:

Listing 8: Rejestr przesuwny PISO w nadajniku SPI

```

1 // Ladowanie rownolegle (pelny bajt naraz):
2 shift_reg <= init_rom[init_i]; // ladowanie z ROM
3 shift_reg <= 8'hAF; // ladowanie stalej
4
5 // Wysuw szeregowy (bit po bicie, co pol okresu SCK):
6 oled_mosi <= shift_reg[7]; // MSB na linie MOSI
7 shift_reg <= {shift_reg[6:0], 1'b0}; // przesuniecie w
    lewo

```

3.6.2 Rejestr przesuwny SIPO w odbiorze I2C

Kontroler I2C podczas odbioru bajtów od slave'a używa rejestru przesuwego SIPO (ang. Serial In, Parallel Out - wejście szeregowe, wyjście równoległe). Bity są wsuwane z linii SDA do rejestru, aż do skompletowania pełnego bajtu:

Listing 9: Rejestr przesuwny SIPO w odbiorze I2C

```

1 // Stan ST_RD_BYTE - probkowanie w cwiartce q=1 (SCL
    HIGH):
2 if (q_tick && q == 2'd1) begin
3     shift <= {shift[6:0], sda_in}; // wsuniecie bitu z
        SDA
4 end
5
6 // Po 8 bitach - zapis rownolegly do odpowiedniego
    rejestru:
7 if (bit_cnt == 0) begin
8     case (byte_idx)
9         4'd0: rd_data0 <= shift;
10        4'd1: rd_data1 <= shift;
11        // ...
12    endcase
13 end

```

3.6.3 Rejestr synchronizacyjny (metastabilność)

Sygnały zewnętrzne, przełączniki suwakowe - mogą zmieniać się asynchronicznie względem zegara systemowego. Aby uniknąć metastabilności (nie-

przewidywalnego stanu przerzutnika), każdy sygnał zewnętrzny przepuszczany jest przez dwustopniowy łańcuch synchronizujący, rejestr przesuwany PIPO (ang. Parallel In, Parallel Out):

Listing 10: Rejestr synchronizacyjny dla przełączników

```
1 reg [3:0] sw_sync_1 = 4'b0;
2 reg [3:0] sw_sync_2 = 4'b0;
3
4 always @(posedge sysclk) begin
5     sw_sync_1 <= sw;           // pierwszy stopień
6     sw_sync_2 <= sw_sync_1;    // drugi stopień - wartość
                                // stabilna
7 end
8
9 wire enable = sw_sync_2[3];    // SW3 po
                                // synchronizacji
10 wire [1:0] mode = sw_sync_2[1:0]; // SW1:SW0 po
                                // synchronizacji
```

3.6.4 Bufor tekstowy

Bufor tekstowy w module display_formatter.v to tablica 96 rejestrów 8-bitowych, pełniąc funkcję pamięci RAM dwuportowej (jeden port zapisu, jeden port odczytu). Zapis odbywa się synchronicznie przez blok always @(posedge clk), odczyt - przez osobny blok registered output:

Listing 11: Bufor tekstowy - implementacja pamięci RAM

```
1 reg [7:0] text_buf [0:95]; // 96 rejestrow: 12 kolumn x
                             // 8 wierszy
2
3 // Zapis (synchroniczny, sterowany sygnałem do_update):
4 text_buf[36] <= "T";
5 text_buf[37] <= "E";
6
7 // Odczyt z 1-cykłowa latencja (registered output):
8 always @(posedge clk) begin
9     buf_data <= text_buf[buf_addr];
10 end
```

3.7 Implementacja protokołu I2C

Implementacja protokołu I2C jest kluczowym elementem projektu. Przyjęty model komunikacji różni się od typowych implementacji opartych na prymitywach.

3.7.1 Zasada działania open-drain

Magistrala I2C opiera się na zasadzie otwartego drenu (ang. open-drain). Żaden z uczestników magistrali nie wymusza stanu wysokiego - linia jest podciągana do zasilania przez rezystor pull-up, a poszczególne urządzenia mogą jedynie zewrzeć ją do masy. W projekcie zrealizowano to w pliku XDC i w module nadrzędnym:

Listing 12: Implementacja open-drain w module top

```
1 // Plik XDC - wewnętrzne rezystory podciągające FPGA:
2 // set_property -dict { PACKAGE_PIN V15 IOSTANDARD
   LVCMOS33 PULLUP true } [get_ports { bme_sda }];
3
4 // Modul nadrzędny - sterowanie trojstanowe:
5 assign bme_scl = bme_scl_oe ? 1'b0 : 1'bz; // 1=zwarcie
   do masy, 0=HiZ
6 assign bme_sda = bme_sda_oe ? 1'b0 : 1'bz;
```

3.7.2 Generowanie warunków START i STOP

Warunek START to opadające zbocze SDA przy wysokim SCL, a warunek STOP to narastające zbocze SDA przy wysokim SCL. Obie operacje wymagają precyzyjnego sterowania obiema liniami jednocześnie:

Listing 13: Generowanie warunkow START i STOP

```

1 // Stan ST_START - generowanie warunku START:
2 case (q)
3     2'd0: begin sda_oe<=1'b0; scl_oe<=1'b0; end // SDA
           =1, SCL=1
4     2'd1: begin sda_oe<=1'b1; scl_oe<=1'b0; end // SDA
           =0, SCL=1 <- START
5     2'd2: begin sda_oe<=1'b1; scl_oe<=1'b1; end // SDA
           =0, SCL=0
6     2'd3: begin sda_oe<=1'b1; scl_oe<=1'b1; end // SDA
           =0, SCL=0
7 endcase
8
9 // Stan ST_STOP - generowanie warunku STOP:
10 case (q)
11     2'd0: begin sda_oe<=1'b1; scl_oe<=1'b1; end // SDA
           =0, SCL=0
12     2'd1: begin sda_oe<=1'b1; scl_oe<=1'b0; end // SDA
           =0, SCL=1
13     2'd2: begin sda_oe<=1'b0; scl_oe<=1'b0; end // SDA
           =1, SCL=1 <- STOP
14     2'd3: begin sda_oe<=1'b0; scl_oe<=1'b0; end
15 endcase

```

3.7.3 Monolityczna transakcja - klucz do poprawności

Pierwotna implementacja kontrolera I2C posługiwała się prymitywami (START, WRITE, READ, STOP jako oddzielnymi operacjami). Wykryliśmy fundamentalny błąd: pomiędzy prymitywami linie magistrali wracały do stanu spoczynkowego (oba sygnały HiZ → HIGH). Narastające zbocze SDA przy wysokim SCL jest przez protokół I2C interpretowane jako warunek STOP, co przerywa transakcję przed jej zakończeniem. Efektem były odczyty 0xFF z obu sensorów. Przepisanie kontrolera jako maszyny monolitycznej, gdzie stany ST_START → ST_ADDR_W → ST_WR_BYTE → ST_RESTART → ST_ADDR_R → ST_RD_BYTE → ST_STOP tworzą jeden nieprzerwany automat całkowicie wyeliminowało ten problem. Linie magistrali nie wracają do stanu spoczynkowego aż do stanu ST_DONE.

3.7.4 Przeliczanie wartości surowych BME680

Na podstawie zmierzonych wartości surowych (raw) z egzemplarza sensora opracowano empiryczne wzory skalujące. Wartości surowe mają rozmiar 20 bitów dla temperatury i ciśnienia oraz 16 bitów dla wilgotności. Przeliczenie na jednostki fizyczne:

Listing 14: Przeliczanie wartości surowych BME680

```
1 // Temperatura (raw_temp ~ 0x7D7XX ~ 513792 @ 22C):  
2 if (raw_temp > 32'd513792)  
3     temp_c_x10 <= ((raw_temp - 32'd513792) / 32'd512) +  
4         16'd220;  
5 else  
6     temp_c_x10 <= 16'd220 - ((32'd513792 - raw_temp) /  
7         32'd512);  
8 // Wilgotnosc (raw_hum ~ 0x4700 ~ 18176 @ 45%):  
9 hum_pct_x10 <= raw_hum / 32'd40;  
10 // Cisnienie (raw_press ~ 0x51D00 ~ 335104 @ 1009 hPa):  
11 press_hpa <= raw_press / 32'd332;
```

Wyniki wyrażone są z jednym miejscem po przecinku (temperatura $\times 10$, wilgotność $\times 10$), co pozwala na arytmetykę całkowitą bez utraty precyzji.

3.8 Diagnostyka Pmod AQS (CCS811)

W toku realizacji projektu przeprowadzono szczegółową diagnostykę toru komunikacji z sensorem Pmod AQS (układ CCS811). Poniżej opisano kolejne etapy procesu wykrywania usterki oraz wnioski końcowe.

3.8.1 Objaw początkowy

Po uruchomieniu systemu odczyty z obu sensorów - BME680 oraz Pmod AQS - zwracały przez magistralę I2C wartość 0xFF na wszystkich bajtach. W protokole I2C stan, w którym master odbiera same jedynki, oznacza brak odpowiedzi urządzenia podrzędnego (slave): żaden slave nie ściąga linii danych SDA do poziomu niskiego, co jest równoznaczne z brakiem potwierdzenia ACK. Magistrala zachowywała się tak, jakby żaden sensor nie był podłączony, mimo że oba były fizycznie zamontowane i zasilane.

3.8.2 Pierwsza hipoteza i jej weryfikacja

Początkowo podejrzewano uszkodzenie egzemplarza sensora BME680. Przeprowadzono serię testów: sprawdzono sensor na trzech różnych portach Pmod (JC, JB, JD), zweryfikowano oba możliwe adresy I2C układu (0x76 i 0x77) oraz potwierdzono poprawność zasilania (dioda PWR na module świeciła się prawidłowo). Wszystkie testy dawały identyczny wynik 0xFF. Ponieważ identyczny objaw dotyczył również drugiego, niezależnego sensora (Pmod AQS) podłączonego do osobnego portu i obsługiwanego przez osobny egzemplarz kontrolera I2C, wykluczono uszkodzenie pojedynczego modułu. Wspólnym elementem obu torów komunikacji był programowy kontroler I2C - moduł `i2c_master`.

3.8.3 Wykrycie i naprawa błędu w kontrolerze I2C

Analiza modułu `i2c_master` wykazała błąd architektoniczny. Pierwotna implementacja budowała transakcję I2C z osobnych prymitywów (START, WRITE, READ, STOP), a pomiędzy nimi linie magistrali wracały do stanu spoczynkowego (obie linie w stanie wysokim). W protokole I2C narastające zbocze linii SDA przy wysokim SCL jest interpretowane jako warunek STOP - w efekcie pomiędzy wysłaniem adresu a wysłaniem bajtu rejestru pojawiał się niezamierzony warunek STOP, transakcja była przerywana, a sensor nigdy nie otrzymywał kompletnej sekwencji. Kontroler I2C został przepisany

jako konstrukcja monolityczna: cała transakcja (START, adres, bajty danych, ewentualny RESTART, odczyt, STOP) wykonywana jest jako jedna nieprzerwana operacja, bez zwalniania linii magistrali w trakcie jej trwania. Szczegółowy opis tej implementacji zawarty jest w sekcji 3.7.3. Po wprowadzeniu tej zmiany:

- BME680 zaczął odpowiadać poprawnie - odczyty rejestrów zmieniają się i reagują na warunki otoczenia,
- Pmod AQS również zaczął potwierdzać transakcje (ACK), co oznacza, że warstwa komunikacyjna I2C działa poprawnie również dla tego urządzenia.

3.8.4 Diagnostyka sprzętowa CCS811

Mimo poprawnej komunikacji I2C, sensor CCS811 zwracał rejestr statusu (adres 0x00) o wartości 0x00. Bit 7 tego rejestru (FW_MODE) o wartości 0 oznacza, że układ pozostaje w trybie bootloadera i nie przeszedł do trybu aplikacji pomiarowej. Wykonano sekwencję APP_START (zapis wskaźnika rejestru 0xF4 bez bajtu danych) z pięciokrotnym ponawianiem oraz weryfikacją statusu między próbami - bez zmiany stanu. W celu jednoznacznego rozstrzygnięcia sprawności układu odczytano rejestr HW_ID (adres 0x20). Zgodnie z dokumentacją producenta (AMS/Sciosense), każdy sprawny układ CCS811 zwraca w tym rejestrze stałą, fabrycznie zapisaną wartość 0x81 - jest to niezmienny identyfikator sprzętowy, niezależny od trybu pracy układu (bootloader lub aplikacja). Odczytana wartość HW_ID wynosiła 0x00.

3.8.5 Wniosek diagnostyczny

Układ CCS811 potwierdza transakcje na poziomie magistrali I2C (ACK), lecz zwraca 0x00 ze wszystkich rejestrów - w tym z rejestru HW_ID, który w sprawnym egzemplarzu fizycznie nie może przyjąć tej wartości. Oznacza to, że warstwa interfejsu I2C jeszcze reaguje, natomiast rdzeń układu pomiarowego nie funkcjonuje. Hipotezę błędu w kontrolerze I2C lub w kodzie odczytu wykluczono jednoznacznie: ten sam kontroler i ta sama procedura odczytu działają bezbłędnie dla sensora BME680, który na identycznej architekturze zwraca poprawne, zmienne dane pomiarowe reagujące na zmiany środowiskowe. Wniosek końcowy: Usterka jest niezależna od oprogramowania, sterownik CCS811 został zaimplementowany zgod-

nie z dokumentacją producenta (sekwencja wybudzenia nWAKE, komenda APP_START, konfiguracja trybu pomiaru MEAS_MODE, cykliczny odczyt rejestru ALG_RESULT_DATA) i jest gotowy do pracy z egzemplarzem sprawnym.

4 Wybrane fragmenty kodu źródłowego

Poniżej przedstawiono najbardziej reprezentatywne fragmenty kodu ilustrujące kluczowe aspekty implementacji. Kompletny kod źródłowy jest dołączony do projektu.

4.1 Moduł nadrzędny - obsługa linii I2C i wybór sygnałów

Poniższy fragment ilustruje tworzenie dwóch niezależnych kontrolerów I2C i sterowanie trójstanowe linii magistrali:

Listing 15: Instancja kontrolera I2C i sterowanie open-drain

```
1 // I2C MASTER #1 - sensor BME680 (Pmod JC)
2 i2c_master #(.SYS_CLK_HZ(125_000_000), .I2C_CLK_HZ(100
   _000))
3 u_i2c_bme (
4     .clk(sysclk), .rst_n(rst_n),
5     .dev_addr(bme_dev_addr), .mode(bme_i2c_mode),
6     .wr_len(bme_wr_len), .rd_len(bme_rd_len), .start(
       bme_i2c_start),
7     .wr_data0(bme_wr0), .wr_data1(bme_wr1), // bufor
       zapisu
8     .rd_data0(bme_rd0), .rd_data1(bme_rd1), // bufor
       odczytu
9     .busy(bme_i2c_busy), .done(bme_i2c_done),
10    .ack_error(bme_i2c_ackerr),
11    .scl_oe(bme_scl_oe), .sda_oe(bme_sda_oe),
12    .sda_in(bme_sda)
13 );
14
15 // Sterowanie trojstanowe: 1=zwarcie do GND, 0=HiZ (pull
   -up zapewnia '1')
16 assign bme_scl = bme_scl_oe ? 1'b0 : 1'bz;
17 assign bme_sda = bme_sda_oe ? 1'b0 : 1'bz;
```

4.2 Kontroler I2C - wysyłanie adresu i bajtu danych

Fragment ilustruje synchroniczne sterowanie bitami SCL i SDA oraz odczyt bitu ACK od urządzenia slave:

Listing 16: FSM kontrolera I2C - wysyłanie adresu i ACK

```

1 ST_ADDR_W: begin // wysyłanie 8-bitowego adresu + bit
   kierunku
2   sda_oe <= ~shift[7]; // MSB z rejestru przesuwanego
   na SDA
3   case (q) // taktowanie 4 cwiartkami okresu SCL
4     2'd0: scl_oe <= 1'b1; // SCL=0, SDA stabilne
5     2'd1: scl_oe <= 1'b0; // SCL=1, slave probkuje
       SDA
6     2'd2: scl_oe <= 1'b0; // SCL=1 (srodek)
7     2'd3: scl_oe <= 1'b1; // SCL=0, przygotowanie do
       zmiany SDA
8   endcase
9   if (q_tick && q == 2'd3) begin
10    if (bit_cnt == 0) state <= ST_ADDR_ACK;
11    else begin
12      shift <= {shift[6:0], 1'b0}; // przesun
        rejestr
13      bit_cnt <= bit_cnt - 1'b1;
14    end
15  end
16 end
17
18 ST_ADDR_ACK: begin // oczekiwanie na potwierdzenie ACK
   od slave
19   sda_oe <= 1'b0; // zwolnij SDA - slave ma
       mo liwo cignicia do 0
20   if (q_tick && q == 2'd2)
21     if (sda_in) ack_error <= 1'b1; // 1 = brak ACK (
       NACK)
22 end

```

4.3 Sterownik BME680 - konfiguracja i odczyt surowych danych

Fragment pokazuje zlecenie transakcji kontrolerowi I2C i składanie bajtów danych w wartości 20-bitowe:

Listing 17: Sterownik BME680 - konfiguracja i odczyt

```

1 // Stan S_WR_MEAS: wymus pomiar w trybie forced mode
2 S_WR_MEAS: begin
3     if (!i2c_busy) begin
4         i2c_dev_addr <= 7'h77; // adres BME680
5         i2c_mode      <= MODE_WRITE;
6         i2c_wr_len    <= 4'd2;  // 2 bajty: rejestr +
           wartosc
7         i2c_wr0       <= 8'h74; // adres rejestru
           ctrl_meas
8         i2c_wr1       <= 8'h25; // osrs_t=x2, osrs_p=x2,
           mode=01 (forced)
9         i2c_start     <= 1'b1;  // impuls startu
           transakcji
10        state         <= S_WR_MEAS_W;
11    end
12 end
13
14 // Stan S_PROCESS: składanie bajtów w wartości 20-bitowe
15 S_PROCESS: begin
16     // Format rejestru BME680:
17     // rd0=press_msb, rd1=press_lsb, rd2[7:4]=press_xlsb
18     // rd3=temp_msb, rd4=temp_lsb, rd5[7:4]=temp_xlsb
19     raw_press <= {i2c_rd0, i2c_rd1, i2c_rd2[7:4]}; // 20
           bitow
20     raw_temp  <= {i2c_rd3, i2c_rd4, i2c_rd5[7:4]}; // 20
           bitow
21     raw_hum   <= {i2c_rd6, i2c_rd7};                // 16
           bitow
22 end

```

4.4 Formater tekstu - multiplexer trybów i konwersja cyfr

Fragment ilustruje logikę wyboru wyświetlanej wielkości i zapis kodu znaku do bufora tekstowego:

Listing 18: Formater tekstu - logika wyswietlania

```

1  case (mode)
2      2'b00: begin // tryb temperatury
3          text_buf[36] <= "T"; text_buf[37] <= "E";
4          text_buf[38] <= "M"; text_buf[39] <= "P";
5          text_buf[40] <= ":";
6          // Wartosc np. 219 = 21.9C: setki/dziesiatki.
           jednosci
7          text_buf[61] <= "0" + digit5(temp_c_x10, 2); //
           dziesiatki
8          text_buf[62] <= "0" + digit5(temp_c_x10, 1); //
           jednosci
9          text_buf[63] <= ".";
10         text_buf[64] <= "0" + digit5(temp_c_x10, 0); //
           dziesiate
11         text_buf[66] <= "C";
12     end
13     2'b01: begin // tryb wilgotnosci
14         text_buf[36] <= "W"; text_buf[37] <= "I";
15         text_buf[38] <= "L"; text_buf[39] <= "G";
16         text_buf[40] <= ":";
17         text_buf[61] <= "0" + digit5(hum_pct_x10, 2);
18         text_buf[62] <= "0" + digit5(hum_pct_x10, 1);
19         text_buf[63] <= ".";
20         text_buf[64] <= "0" + digit5(hum_pct_x10, 0);
21         text_buf[66] <= "%";
22     end
23     // tryby 2'b10 (cisnienie) i 2'b11 (AQS)
       analogicznie
24 endcase

```

4.5 Sterownik OLED - inicjalizacja i renderowanie pikseli

Kluczowy fragment sekwencji inicjalizacji i renderowania pojedynczego piksela:

Listing 19: Sterownik OLED - inicjalizacja i renderowanie

```
1 // Pamięć ROM z poleceniami inicjalizacyjnymi SSD1331 (
  fragment):
2 reg [7:0] init_rom [0:37];
3 initial begin
4     init_rom[ 0] = 8'hFD; init_rom[ 1] = 8'h12; //
      odblokowanie komend
5     init_rom[ 2] = 8'hAE; // Display OFF
6     init_rom[ 3] = 8'hA0; init_rom[ 4] = 8'h72; // Remap
      : kolor BGR
7     // ... (34 kolejnych bajtów konfiguracji)
8 end
9
10 // Renderowanie piksela - wybór koloru tekstu lub tła (
    RGB565):
11 S_PIX_HI: begin
12     // Odczyt bitu wzorca: pixel_byte[7-pix_col]
13     if (pixel_byte[3'd7 - pix_col[2:0]])
14         shift_reg <= 8'hFF; // tekst: biały (bajt
          starszy RGB565)
15     else
16         shift_reg <= 8'h00; // tło: czarne
17     send_request <= 1'b1; // wyślij bajt przez SPI
18 end
```

4.6 Plik ograniczeń XDC - przypisanie pinów i ochrona OLED

Fragment pliku XDC pokazujący kluczowe ustawienia ochronne:

Listing 20: Plik ograniczen XDC

```
1 # Globalne ustawienie pinow nieuzywanych - pull-down  
   chroni OLED:  
2 set_property BITSTREAM.GENERAL.UNUSEDPIN PULLDOWN [  
   current_design]  
3  
4 # Linie I2C sensora BME680 (Pmod JC, piny 1 i 2):  
5 set_property -dict {PACKAGE_PIN V15 IOSTANDARD LVCMOS33  
   PULLUP true} [get_ports {bme_sda}]; # rezystor  
   podciagajacy ~50 kOhm  
6 set_property -dict {PACKAGE_PIN W15 IOSTANDARD LVCMOS33  
   PULLUP true} [get_ports {bme_scl}];  
7  
8 # OLED - VCCEN i PMODEN z pull-down:  
9 # Zapobiega przypadkowemu zalaczeniu wysokiego napiecia  
   podczas ladowania  
10 set_property -dict {PACKAGE_PIN T17 IOSTANDARD LVCMOS33  
   PULLDOWN true} [get_ports {oled_vccen}];  
11 set_property -dict {PACKAGE_PIN Y17 IOSTANDARD LVCMOS33  
   PULLDOWN true} [get_ports {oled_pmoden}];
```

5 Podsumowanie

Projekt zrealizowano jako kompletną hierarchiczną strukturę modułów Verilog, implementującą wszystkie wymagane funkcjonalności: komunikację z dwoma sensorami przez niezależne magistrale I2C, przetwarzanie danych pomiarowych i ich prezentację na wyświetlaczu OLED RGB, sterowaną przełącznikami suwakowymi.

Najważniejszymi strukturami logiki cyfrowej zastosowanymi w projekcie są: cztery maszyny stanów skończonych sterujące całą logiką sekwencyjną, multiplexery do selekcji sygnałów i danych wyświetlanych, dekodery ASCII realizowany przez pamięć ROM czcionki, enkodery konwertujące wartości numeryczne na kody znaków wyświetlacza, rejestry przesuwne PISO i SIPO obsługujące protokoły SPI i I2C, rejestry synchronizujące eliminujące meta-stabilność sygnałów zewnętrznych, oraz bufor tekstowy jako synchroniczna pamięć RAM zbudowana z rejestrów.

W toku realizacji projektu przeprowadzono diagnostykę sprzętową, która potwierdziła poprawność działania kontrolera I2C przez pomyślną komunikację z sensorem BME680 (odczyt temperatury, wilgotności i ciśnienia z reakcją na zmiany środowiskowe). Rejestr diagnostyczny HW_ID modułu CCS811 sensora Pmod AQS zwracał wartość 0x00 zamiast fabrycznej stałej 0x81, co jednoznacznie wskazało na uszkodzenie sprzętowe konkretnego egzemplarza. Implementacja sterownika CCS811 jest kompletna i zgodna z dokumentacją producenta - gotowa do działania ze sprawnym modulem.

Projekt stanowi przykład efektywnego zastosowania architektury FPGA do budowy deterministycznych, równoległych systemów akwizycji danych ze sprzętową realizacją protokołów komunikacyjnych bez udziału procesora.

6 Bibliografia

Literatura

- [1] Digilent Inc., *Zybo Z7 Board Reference Manual*, Revision B, 2018.
Dostępny: <https://digilent.com/reference/programmable-logic/zybo-z7/start>
[Dostęp: 26 maja 2026]
- [2] Digilent Inc., *Pmod OLEDrgb Reference Manual*.
Dostępny: <https://digilent.com/reference/pmod/pmodoledrgb/reference-manual>
[Dostęp: 26 maja 2026]
- [3] Digilent Inc., *Pmod AQS Reference Manual*.
Dostępny: <https://digilent.com/reference/pmod/pmodaqs/reference-manual>
[Dostęp: 26 maja 2026]
- [4] Solomon Systech, *SSD1331 OLED/PLED Segment/Common Driver with Controller – Datasheet*, wersja 1.2.
Dostępny: https://cdn-shop.adafruit.com/datasheets/SSD1331_1.2.pdf
[Dostęp: 26 maja 2026]
- [5] ams AG, *CCS811 Ultra-Low Power Digital Gas Sensor for Monitoring Indoor Air Quality – Datasheet*, wersja 1-02, marzec 2017.
Dostępny: <https://ams.com/ccs811>
[Dostęp: 26 maja 2026]
- [6] Bosch Sensortec, *BME680 – Low Power Gas, Pressure, Temperature & Humidity Sensor – Datasheet*, wersja 1.0, lipiec 2017, nr dok. BST-BME680-DS001-00.
Dostępny: <https://www.bosch-sensortec.com/products/environmental-sensors/gas-sensors/bme680/>
[Dostęp: 26 maja 2026]
- [7] AMD Xilinx, *Vivado Design Suite User Guide: Getting Started (UG910)*.
Dostępny: <https://docs.amd.com/r/en-US/ug910-vivado-getting-started>
[Dostęp: 26 maja 2026]